GoDHCP

Руководство администратора

Версия 0.1.8 · Linux amd64 и arm64

© 2026 Кислов Роман Сергеевич. Apache License 2.0.

Установка пакетов и запуск через systemd. Экспорт настроек ISC DHCP, импорт, проверка переноса, переключение и откат. Резервное копирование и диагностика.

Перед переключением рабочей сети пройдите приёмку на изолированном стенде. Действующие аренды ISC автоматически не переносятся.

Содержание

Состояние текущей версии 3

Установка пакета для дистрибутива 4

Ручная установка бинарного файла 4

Экспорт настроек из ISC DHCP 5

- Найти действующую конфигурацию . . . 5
- Сохранить исходные файлы . . . 5
- Преобразовать экспорт в YAML GoDHCP . . . 6
- Проверить полноту переноса . . . 6

Настройка GoDHCP на стенде 7

Запуск через systemd 9

Работа с веб-интерфейсом 10

Приёмка и переключение с ISC DHCP 10

- Откат . . . 11

Изменение настроек 11

Резервное копирование и восстановление 12

Повторяемый сетевой стенд в Docker 12

Проверка systemd с настоящей перезагрузкой Linux 13

Ежедневная проверка и диагностика 13

Лицензия и автор 14

Обновление до v0.1.8 и повторный импорт 14

- 1. Проверьте версию и действующие пути . . . 14
- 2. Скачайте пакет и проверьте SHA-256 . . . 14
- 3. Остановите службу и сохраните настройки и аренды . . . 15
- 4. Установите обновление . . . 15
- 5. Повторите импорт в отдельный файл . . . 16
- 6. Перенесите недостающие резервации в копию рабочего конфига . . . 16
- 7. Проверьте кандидат и запустите сервер . . . 17
- 8. Проверьте результат в веб-интерфейсе . . . 17
- 9. Откат при проблеме . . . 18

© 2026 Кислов Роман Сергеевич. Apache License 2.0.

Установка и обслуживание DHCPv4-сервера на Linux, экспорт настроек ISC DHCP (`isc-dhcp-server` / `dhcpd`) и подготовка перехода на GoDHCP. Примеры рассчитаны на Debian/Ubuntu с systemd. Адреса, имена интерфейсов и пути необходимо заменить на свои. Команды администрирования выполняются в root-сессии Bash (`sudo -i`).

Для ежедневной работы в браузере см. пошаговую инструкцию по веб-интерфейсу (web-guide.md): вход, поиск устройства, постоянный IP для принтера и проверка результата.

Состояние текущей версии

В версии 0.1.1 исправлены четыре ошибки DHCP-ядра: OFFER/ACK отправляется только после успешного сохранения аренды, REQUEST не меняет подсеть по запрошенному IP при известном входном VLAN, RELEASE/DECLINE проверяют владельца действующей аренды, повторный DISCOVER сохраняет состояние и срок bound-аренды. Добавлены регрессионные тесты. Выдача, reload и очистка аренд согласованы внутри одного процесса.

В версии 0.1.2 исправлены ещё четыре сетевые ошибки: начальные OFFER/ACK доставляются клиентам без broadcast-флага, прямой NAK всегда отправляется broadcast с нулевым ciaddr, BOOTREPLY игнорируется, явное имя интерфейса из `vlans[].interface` участвует в выборе VLAN. Ошибки отправки UDP теперь попадают в журнал. Для начальных ответов используется разрешённый RFC 2131 §4.1 broadcast fallback: обычный UDP-сокет не умеет отправлять кадр непосредственно на MAC ещё не настроенного клиента. Продление остаётся unicast, ответы relay идут на giaddr:67.

В версии 0.1.3 запуск systemd переведён на `Type=notify`: готовность подтверждается после открытия DHCP, HTTP, metrics и управляющего сокета. Ошибка bind завершает процесс с ненулевым кодом; `Restart=on-failure` позволяет повторить запуск. Reload через приватный Unix-сокет ждёт применения конфигурации и возвращает ошибку при некорректном YAML. Стартовые настройки больше не перезаписываются из обработчика SIGHUP.

На изолированном Docker-стенде Linux ARM64 выполнена 21 проверка: получение и продление адресов, настоящий 802.1Q, ISC relay с Option 82, автоматическое продление ISC dhclient, перезапуск процесса и пересоздание контейнера с сохранённой SQLite. Повторный запуск описан ниже. Испытания не означают совместимость с ISC failover или проверку всех сетевых сценариев; перед заменой рабочего DHCP нужны приёмочные испытания в вашей сети.

Дополнительно учитывайте:

- `ping_check` не подключён в точке запуска; включение параметра пока не защищает от конфликтов IP.
- Создание чистой базы PostgreSQL содержит ошибки миграции; для стенда используйте SQLite.
- `raw_vlan` не подключён. Для локальных VLAN нужны заранее созданные ОС интерфейсы. PCP на Linux работает через SO_PRIORITY и заранее настроенную egress-qos-map; см. настройку PCP (vlans.md#pcp-на-linux).
- `server.interfaces` не является ограничением приёма UDP-пакетов. Ограничивайте доступ сетевыми правилами и выбором адреса `server.listen`.
- `/readyz` проверяет базу, но не прохождение DHCP через сеть. В systemd готовность при запуске дополнительно требует открытия всех слушателей.
- DHCPv6, DDNS и обмен состоянием ISC failover не реализованы. `ha` делит клиентов по хэшу; два экземпляра с пересекающимися пулами могут выдать одинаковый IP.

Ниже приведена процедура подготовки и стендовой проверки. Переключение рабочей сети выполняйте после оценки оставшихся ограничений и приёмочных испытаний.

Установка пакета для дистрибутива

Подготовлены пакеты версии 0.1.3 для Linux amd64 и arm64. Для Debian/Ubuntu используйте `.deb`, для RHEL, Rocky Linux, AlmaLinux и Fedora - `.rpm`. Это статический Go-бинарный файл без зависимости от glibc; установка пакетов проверяется отдельно от работы DHCP в реальной сети. Архив `.tar.gz` предназначен для ручной установки, включая другие Linux-дистрибутивы.

На машине со сборками проверьте контрольные суммы, затем передайте пакет нужной архитектуры на сервер:

```
cd dist/releases/0.1.3
sha256sum -c SHA256SUMS
```

Debian/Ubuntu, amd64:

```
apt install ./godhcp_0.1.3_linux_amd64.deb
```

RHEL-подобные системы/Fedora, amd64:

```
dnf install ./godhcp_0.1.3_linux_amd64.rpm
```

Для ARM64 выберите файл с `arm64`; внутри RPM эта архитектура обозначена как `aarch64`. Архитектуру хоста проверьте через `uname -m`: `x86_64` соответствует amd64, `aarch64` соответствует arm64.

Пакет устанавливает `/usr/bin/godhcp`, unit `/usr/lib/systemd/system/godhcp.service`, пример `/etc/godhcp/config.yml.example`, документацию и PDF в `/usr/share/doc/godhcp/`. Скрипт установки создаёт системного пользователя `godhcp` и каталоги. Служба не запускается и не включается автоматически; активный конфиг и секреты не создаются. При обновлении существующая служба автоматически не перезапускается, поэтому новый бинарный файл начнёт обслуживать сеть после запланированного перезапуска.

При новой установке скопируйте пример, если ещё не подготовлен импортированный конфиг:

```
test -f /etc/godhcp/config.yml || \
  install -m 0640 -o godhcp -g godhcp \
  /etc/godhcp/config.yml.example /etc/godhcp/config.yml
```

Настройте адреса, интерфейсы и аутентификацию по разделам ниже. В пакетном примере DHCP отключён через `server.listen: "-"`; замените значение на `":67"` перед сетевыми испытаниями. Затем проверьте конфиг и выполните запуск через systemd.

Ручная установка бинарного файла

На машине сборки нужен Go 1.26 или новее. В корне проекта:

```
go version
go test ./...
go vet ./...
make build
./bin/godhcp version
```

Для Linux amd64 при сборке на другой ОС:

```
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -trimpath -o godhcp-linux-amd64 ./cmd/godhcp
```

Если выбран пакет, пропустите ручную установку и создание пользователя. Для архива распакуйте его в отдельный каталог: `tar -xzf godhcp_0.1.3_linux_amd64.tar.gz`. В архиве бинарный файл называется `godhcp`, рядом лежат unit, пример конфига, документация и PDF. На целевом Linux установите подходящий бинарный файл и создайте каталоги:

```
# Для бинарного файла из архива укажите godhcp вместо godhcp-linux-amd64.
install -m 0755 godhcp-linux-amd64 /usr/bin/godhcp
useradd --system --home-dir /var/lib/godhcp --shell /usr/sbin/nologin godhcp
install -d -m 0750 -o godhcp -g godhcp /etc/godhcp /var/lib/godhcp
```

Если пользователь уже существует, повторно создавать его не нужно. `version` выводит версию и завершает работу; DHCP-порт не открывается.

Экспорт настроек из ISC DHCP

Найти действующую конфигурацию

У ISC DHCP конфигурация хранится в текстовом `dhcpd.conf` и подключаемых файлах `include`. Для переноса нужен этот набор файлов; база `dhcpd.leases` хранит выданные аренды отдельно. Параметры `-cf` и `-lf` задают альтернативные пути. См. руководство ISC dhcpd.

На действующем сервере выполните:

```
systemctl status isc-dhcp-server --no-pager
systemctl cat isc-dhcp-server
ps -ww -C dhcpd -o args=
```

В некоторых дистрибутивах служба называется `dhcpd`: замените имя в командах. Зафиксируйте основной конфиг, путь аренд, интерфейсы, chroot и способ запуска. В Debian/Ubuntu обычно используются `/etc/dhcp/dhcpd.conf`, `/var/lib/dhcp/dhcpd.leases` и `/etc/default/isc-dhcp-server`; фактические параметры процесса имеют приоритет.

Просмотрите включения, затем откройте найденные файлы и их вложенные включения:

```
rg -n '^\s*include\s+' /etc/dhcp
```

Если `rg` не установлен, используйте `grep -R -n 'include' /etc/dhcp`. Поиск служит подсказкой: проверьте также абсолютные пути вне `/etc/dhcp`, ссылки и файлы внутри chroot. Запретите параллельное изменение конфигурации на время экспорта.

Сохранить исходные файлы

Следующий пример архивирует `/etc/dhcp`, не останавливая действующий DHCP:

```
umask 077
export_dir="/root/isc-dhcp-export-$(date +%Y%m%d-%H%M%S)"
install -d -m 0700 "$export_dir"
tar -czf "$export_dir/settings.tar.gz" -C / etc/dhcp
systemctl cat isc-dhcp-server > "$export_dir/service.txt"
ps -ww -C dhcpd -o args= > "$export_dir/process.txt"
if test -f /etc/default/isc-dhcp-server; then
  cp -a /etc/default/isc-dhcp-server "$export_dir/isc-dhcp-server.defaults"
fi
if test -f /var/lib/dhcp/dhcpd.leases; then
  cp -a /var/lib/dhcp/dhcpd.leases "$export_dir/dhcpd.leases.snapshot"
fi
sha256sum "$export_dir/settings.tar.gz" > "$export_dir/settings.sha256"
```

Если включения находятся вне `/etc/dhcp`, включите их в команду `tar` отдельными путями относительно `/`, например `etc/dhcp opt/company/dhcp`. Архив должен содержать каждый реально используемый файл. Скопируйте отдельно внешние цели символических ссылок, если они не попали в архив. Для службы `dhcpd` скорректируйте и команды, сохраняющие описание службы.

Копия `dhcpd.leases` работающего сервера - предварительный снимок для анализа. Финальную копию для отката сохраните после остановки ISC DHCP в согласованное окно. Файлы экспорта могут содержать ключи TSIG, адреса и MAC клиентов: храните их с ограниченным доступом.

Преобразовать экспорт в YAML GoDHCP

Предпочтительный вариант - выполнить импорт на исходном сервере, где доступны все исходные пути. Для этого достаточно скопировать бинарный файл GoDHCP: команда `import-dhcpd` не запускает DHCP-сервер и не меняет исходные файлы.

Сначала проверьте исходный конфиг самим ISC DHCP. Ключ `-t` проверяет синтаксис без сетевого обслуживания:

```
dhcpd -4 -t -cf /etc/dhcp/dhcpd.conf
```

Затем выполните преобразование, используя каталог экспорта из предыдущего шага:

```
/usr/bin/godhcp import-dhcpd \
  -in /etc/dhcp/dhcpd.conf \
  -out "$export_dir/godhcp-imported.yml" \
  2> "$export_dir/import.log"
```

Проверьте код завершения сразу после команды (`echo $?`), затем прочитайте **весь** `import.log`. При ненулевом коде не используйте результат. Строки `read:` перечисляют прочитанные файлы; `warning:` указывают на потери или ограничения переноса. Сравните этот перечень с составом архива. Нулевой код означает, что преобразование удалось, но не гарантирует эквивалентность поведения серверов.

```
cat "$export_dir/import.log"
/usr/bin/godhcp -validate -config "$export_dir/godhcp-imported.yml"
```

Если импорт выполняется на отдельной машине, сохраните структуру каталогов. Относительные `include` GoDHCP ищет относительно файла, содержащего включение; абсолютные пути продолжат ссылаться на исходные абсолютные адреса файлов. Распаковывайте архив в изолированной машине или контейнере с нужными путями либо исправьте пути в рабочей копии. Не распаковывайте чужой `/etc/dhcp` поверх действующей конфигурации другого сервера.

Импорт раскрывает вложенные включения до 32 уровней, отвергает циклы и пропускает повторное включение одного файла. Если ISC использует повторное включение в разных областях или относительные пути с другой базой разрешения, сравните результат вручную.

Проверить полноту переноса

Автоматически переносятся IPv4-подсети, первый диапазон `range` каждой подсети, резервации `hardware ethernet` + `fixed-address`, поддерживаемые опции, сроки аренды, `next-server` и `filename`. `group` и `shared-network` переносят часть общих параметров в подсети; семантика общей физической сети автоматически не восстанавливается.

Обязательно проверьте:

- **Все диапазоны.** Второй и последующие `range` не переносятся. Не объединяйте диапазоны через занятые адреса; модель текущей версии допускает один динамический диапазон на подсеть. `range dynamic-bootp` требует ручной обработки.
- **Опции и наследование.** Сверьте шлюз, DNS, домен, NTP, PXE и vendor options для каждой подсети. Глобальные опции и локальные переопределения могут преобразовываться иначе; глобальные `next-server` и `filename` не следует считать перенесёнными без проверки.
- **Резервации.** Сверьте количество, MAC, IP и принадлежность подсети. Из списка `fixed-address` берётся первый адрес; имена вместо IPv4 требуют ручной обработки. Настройки внутри `host`, кроме поддержанных полей, могут быть потеряны.
- **Политики.** `pool` с allow/deny, `class`/`subclass`, условия и выражения не преобразуются в эквивалентные правила GoDHCP. Часть конструкций пропускается с предупреждением; вручную сопоставьте исходные блоки с YAML.

- **Failover и DDNS.** Настройки ISC failover, TSIG и динамические DNS-обновления не переносятся как работающие функции.
- **Сеть.** Задайте реальные интерфейсы, VLAN, доверенные relay, `giaddr`, Circuit ID и `server_id`. Импорт не восстанавливает сетевую топологию. `server_id` должен быть адресом самого DHCP-сервера, доступным клиентам для продления; автоматически выбранный адрес шлюза может не подходить.
- **Доступ к управлению.** Импортированный YAML не содержит пользователей и API-ключей. До запуска настройте аутентификацию: один `jwt_secret` не закрывает API.

Действующие аренды из `dhcpd.leases` **не импортируются.** YAML переносит конфигурацию, а не состояние выдачи. Нельзя запускать новый сервер с пустой базой и тем же пулом поверх ещё действующих аренд: он может повторно выдать занятые IP. Нужен отдельный план сохранения занятости либо проверенный свободный диапазон, исключённый из выдачи ISC. Автоматической команды импорта аренд в этой версии нет.

Передайте на целевую машину исходный архив, `godhcp-imported.yml` и `import.log` защищённым способом. Сохраните исходную копию YAML, редактируйте отдельный рабочий файл `/etc/godhcp/config.yml`.

Настройка GoDHCP на стенде

Для первого испытания используйте один заранее настроенный VLAN или одну изолированную подсеть. Не подключайте стенд к рабочему широковещательному сегменту DHCP.

Пример `/etc/godhcp/config.yml` для интерфейса `eth0`, которому ОС уже назначила `192.168.50.2/24`:

```
server:
  listen: ":67"
  server_id: 192.168.50.2
  interfaces:
    - name: eth0
      mode: access
      vlan: 50
  authoritative: true
  ping_check: false
  manage_links: false
  lease_default: 1h
  lease_max: 24h
database:
  driver: sqlite
  dsn: /var/lib/godhcp/leases.db
api:
  listen: "127.0.0.1:8181"
  auth:
    jwt_secret: "${JWT_SECRET}"
    users:
      - username: admin
        password_hash: "${GODHCP_ADMIN_HASH}"
        role: admin
web:
  enabled: true
  path: /ui
vlans:
  - id: 50
    name: lab
    interface: eth0
    subnet_ref: lab
subnets:
  - id: lab
    network: 192.168.50.0/24
    range: [192.168.50.100, 192.168.50.150]
    gateway: 192.168.50.1
    dns: [192.168.50.1]
    vlan: 50
relay:
  enabled: false
metrics:
  enabled: true
  listen: "127.0.0.1:9090"
logging:
  level: info
  format: json
```

Создайте отдельный секрет JWT и хэш пароля. Команда хэширования принимает пароль аргументом процесса; выполняйте её на доверенной административной машине. Ввод через `read` исключает пароль из истории команд оболочки.

```
umask 077
read -r -s -p 'Пароль администратора: ' admin_password
printf '\n'
admin_hash=$(/usr/bin/godhcp hash-password -password "$admin_password")
unset admin_password
test -n "$admin_hash" || exit 1
printf 'JWT_SECRET=%s\n' "$(openssl rand -hex 32)" > /etc/godhcp/godhcp.env
printf "GODHCP_ADMIN_HASH='%s'\n" "$admin_hash" >> /etc/godhcp/godhcp.env
unset admin_hash
chown root:godhcp /etc/godhcp/godhcp.env
chmod 0640 /etc/godhcp/godhcp.env
chown godhcp:godhcp /etc/godhcp/config.yml
chmod 0640 /etc/godhcp/config.yml
set -a
. /etc/godhcp/godhcp.env
set +a
/usr/bin/godhcp -validate -config /etc/godhcp/config.yml
```

Одинарные кавычки вокруг хэша в env-файле сохраняют символы `$` при чтении оболочкой. Файл подходит также для `EnvironmentFile` systemd. Все переменные `${...}` должны быть доступны при валидации и запуске.

Для trunk создайте VLAN-интерфейсы средствами ОС и настройте постоянное восстановление после перезагрузки; сопоставьте их с `vlans` и `subnets`. Для relay включите `relay.enabled`, укажите доверенные агенты и маршрутизацию подсетей по руководству relay (relay.md). Параметры интерфейсов и firewall подбираются под реальную сеть.

Запуск через systemd

При пакетной установке unit уже находится в `/usr/lib/systemd/system/godhcp.service`; проверьте его через `systemctl cat godhcp`. При ручной установке используйте `deploy/systemd/godhcp.service` из исходников или `godhcp.service` из архива, либо создайте `/etc/systemd/system/godhcp.service` со следующим содержимым:

```
[Unit]
Description=GoDHCP IPv4 server
Wants=network-online.target
After=network-online.target

[Service]
Type=notify
NotifyAccess=main
RuntimeDirectory=godhcp
RuntimeDirectoryMode=0700
User=godhcp
Group=godhcp
WorkingDirectory=/var/lib/godhcp
EnvironmentFile=/etc/godhcp/godhcp.env
ExecStartPre=/usr/bin/godhcp -validate -config /etc/godhcp/config.yml
ExecStart=/usr/bin/godhcp -config /etc/godhcp/config.yml -control-socket /run/godhcp/control.sock
ExecReload=/usr/bin/godhcp reload -socket /run/godhcp/control.sock
TimeoutStartSec=30
TimeoutStopSec=15
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
NoNewPrivileges=true
UMask=0027
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

Этот unit рассчитан на SQLite, отключённый ping-check и интерфейсы, созданные ОС. Он запускает процесс от `godhcp` и даёт только `CAP_NET_BIND_SERVICE` для открытия порта 67. В `systemctl show godhcp` проверьте `Type=notify`, `User=godhcp`, `NoNewPrivileges=yes` и набор capabilities. Секреты в env-файле остаются root:root 0600; systemd читает его до смены пользователя. SQLite и YAML должны быть доступны пользователю godhcp; остальным пользователям чтение секретов и БД не требуется. Для управления VLAN через API понадобятся дополнительные права и отдельно проверенная настройка `CAP_NET_ADMIN`.

Убедитесь, что в стендовом сегменте нет другого DHCP-сервера, затем:

```
systemctl daemon-reload
systemctl start godhcp
systemctl status godhcp --no-pager
journalctl -u godhcp -n 100 --no-pager
ss -lunp 'sport = :67'
curl --fail http://127.0.0.1:8181/healthz
curl --fail http://127.0.0.1:8181/readyz
```

Проверьте в `ss`, что порт принадлежит именно GoDHCP. Сообщение `dhcp listening` и HTTP 200 сами по себе не подтверждают работу DHCP. После успешной приёмки включите автозапуск: `systemctl enable godhcp`.

Работа с веб-интерфейсом

Откройте `http://127.0.0.1:8181/ui/`, войдите как `admin` с выбранным паролем. Интерфейс на Tailwind CSS встроен в бинарник, включая стили и иконки; подключение к CDN не требуется. По умолчанию сеанс хранится до закрытия вкладки; «Запомнить вход» сохраняет его на устройстве. Сменить светлую/тёмную тему можно кнопкой в верхней панели.

- **Обзор** показывает занятость пулов, подсети, VLAN, время работы и выборку аренд. Зелёный индикатор «API доступен» означает доступность управления, а не проверку сетевого DHCP-порта.
- **Аренды**: поиск по IP, MAC и имени, фильтры VLAN/состояния, размер страницы и переход между страницами. Кнопка CSV экспортирует текущую выборку страницы. Удаление требует подтверждения; устройство может использовать IP до конца аренды, даже если запись удалена.
- **Подсети, резервации, VLAN, relay**: таблицы и раскрывающиеся формы добавления. При создании резервации укажите MAC либо Client ID в hex-формате, IP и подсеть. Для Circuit ID доступен тест парсера без изменения настроек.
- **Конфигурация**: YAML-редактор, проверка перед применением, подтверждение сохранения, экспорт и перечитывание файла. Экспорт скрывает JWT и API-ключи и не заменяет резервную копию секретов. Не применяйте YAML из недоверенных источников.
- **Журнал**: поток SSE, поиск, пауза, автоматическая прокрутка и очистка экрана. В браузере удерживается до 1000 событий; очистка экрана не удаляет серверные журналы.
- **Пользователи**: роли admin/operator/viewer. Кнопки и разделы показываются с учётом роли; доступ дополнительно проверяется сервером. UI блокирует удаление последнего администратора, но при прямом редактировании конфигурации всё равно проверяйте, что учётные данные остались.

На телефоне навигация открывается кнопкой меню. Широкие таблицы прокручиваются горизонтально. Кнопка обновления в верхней панели перечитывает данные текущего раздела.

Для удалённого управления используйте SSH-туннель с рабочей станции:

```
ssh -L 18080:127.0.0.1:8181 admin@dhcp-host
```

После этого откройте `http://127.0.0.1:18080/ui/`. API и метрики в примере доступны только через loopback; при публикации на другом адресе настройте TLS и сетевые ограничения.

Приёмка и переключение с ISC DHCP

1. Используйте исправленную версию 0.1.3 или новее и выполните тестирование с учётом оставшихся ограничений. Подготовьте резервные копии и окно работ, подтвердите способ удалённого доступа при отказе DHCP.
2. Сверьте исходный ISC-конфиг и YAML: подсети, диапазоны, резервации, сроки, DHCP options, PXE и relay. Для каждой активной аренды определите, как её IP будет исключён из повторной выдачи.
3. Проверьте на стенде полный обмен DISCOVER → OFFER → REQUEST → ACK, продление, резервации, исчерпание пула, каждый VLAN и каждый relay. Выполните перезапуск с сохранённой базой и убедитесь, что выданные адреса остаются занятыми.

4. В окно переключения остановите ISC DHCP на всех узлах, способных отвечать в переносимом сегменте, включая failover-партнёра. Сохраните финальный снимок аренд. При переносе на другой хост измените DHCP relay / IP helper на сетевом оборудовании по согласованной схеме.
5. Запустите GoDHCP с подготовленной базой и безопасным пулом. Проверьте порт, журнал и реальную выдачу тестовому клиенту в каждом сегменте. Сверьте IP, маску, шлюз, DNS, server-id и время аренды.
6. Наблюдайте за продлениями старых клиентов и отсутствием конфликтов IP. Включайте автозапуск нового сервера после приёмки.

Не запускайте ISC DHCP и GoDHCP одновременно в одном сегменте с пересекающимися пулами. Для диагностики обмена используйте:

```
tcpdump -ni eth0 -vvv 'udp port 67 or udp port 68'
```

При необходимости замените `eth0` на VLAN-интерфейс; захват может содержать идентификаторы клиентов.

Откат

Сначала остановите GoDHCP: `systemctl stop godhcp`. Верните настройки relay и прежний сервис ISC DHCP, используя сохранённый конфиг. До запуска ISC учтите адреса, уже выданные GoDHCP: старая база ISC о них не знает. При непересекающихся пулах оставьте новый диапазон исключённым; при пересечении потребуется согласование занятости. Простое восстановление старого файла аренд не обеспечивает безопасный откат после новых выдач.

После проверки пула запустите прежнюю службу (`systemctl start isc-dhcp-server` либо `dhcpd`) и подтвердите выдачу клиенту. Сохраните базу и журнал GoDHCP для разбора причины отката.

Изменение настроек

Перед изменением сохраните рабочий YAML и секреты. Проверяйте конфиг с теми же переменными окружения, что использует служба:

```
set -a
. /etc/godhcp/godhcp.env
set +a
/usr/bin/godhcp -validate -config /etc/godhcp/config.yml
systemctl reload godhcp
journalctl -u godhcp -n 50 --no-pager
```

Reload обновляет подсети, пулы и часть логики в памяти; UDP/HTTP-слушатели, подключение БД и настройки логирования не пересоздаются. Для изменения listen, TLS, базы, env-файла или журналирования нужен перезапуск. В unit версии 0.1.3 `systemctl reload` вызывает `godhcp reload` через `/run/godhcp/control.sock` и ожидает результата применения в работающем процессе. При ошибке YAML или применения команда завершается ненулевым кодом; для ошибок проверки YAML прежняя конфигурация остаётся активной. Прямой SIGHUP по-прежнему поддерживается, но не сообщает вызывающему процессу результат. Управляющий сокет имеет права 0600 в каталоге 0700, принадлежащем службе; его создаёт и удаляет systemd через RuntimeDirectory.

В Docker/Kubernetes ConfigMap и mount `:ro` нельзя сохранять из UI. Текущий API сначала применяет конфиг в памяти и только затем пишет файл; ошибка записи может оставить новое состояние активным. При таком развёртывании меняйте конфиг снаружи и перезапускайте экземпляр. Helm-шаблоны сейчас используют `emptyDir` для SQLite: замените его постоянным томом до эксплуатации, иначе пересоздание Pod потеряет аренды.

Резервное копирование и восстановление

Сохраняйте вместе бинарный файл или его точную версию, `/etc/godhcp/config.yml`, `/etc/godhcp/godhcp.env` и базу аренд. Экспорт `/api/v1/config` маскирует JWT и API-ключи; для полной копии нужен исходный файл и все внешние секреты.

Для SQLite установите утилиту `sqlite3`. Онлайн-копию создавайте через `.backup`, чтобы учесть WAL; обычный `cp leases.db` во время работы не гарантирует полноту. См. CLI SQLite и описание online backup.

```
umask 077
backup_dir="/root/godhcp-backup-$(date +%Y%m%d-%H%M%S)"
install -d -m 0700 "$backup_dir"
cp -a /etc/godhcp/config.yml /etc/godhcp/godhcp.env "$backup_dir/"
sqlite3 /var/lib/godhcp/leases.db ".backup '$backup_dir/leases.db'"
sqlite3 "$backup_dir/leases.db" 'PRAGMA integrity_check;'
```

Ожидаемый результат `integrity_check` - `ok`. На время получения согласованного комплекта запретите изменения конфигурации; для точного снимка на один момент остановите службу перед копированием. Перенесите резервную копию на отдельный носитель с ограниченным доступом.

При восстановлении остановите службу, сохраните весь текущий каталог БД вместе с `-wal` и `-shm`, затем установите проверенную копию в чистый каталог. Не подмешивайте старые WAL-файлы к восстановленной БД. Восстановите YAML, env-файл и владельца `godhcp:godhcp`, проверьте конфиг и права, запустите службу. Учитывайте аренды, выданные после снимка: восстановление старой БД не делает эти IP свободными.

Повторяемый сетевой стенд в Docker

Из корня исходного проекта на машине с Go и работающим Docker выполните:

```
make test-dhcp-docker
```

Скрипт собирает Linux-бинарник под архитектуру Docker daemon и образ Debian 12 с независимым Scapy-клиентом, ISC dhclient и ISC dhcrelay. Требуются Docker с поддержкой privileged-контейнеров, network namespace и 802.1Q; при сборке образа нужен доступ к Debian-репозиториям. Docker Desktop на ARM64 проверен; на amd64 скрипт выбирает amd64-сборку, но отдельный сетевой прогон на amd64 пока не выполнен.

Стенд работает с `--network none`: veth, namespace, VLAN 20/30 и relay создаются внутри одноразового контейнера. UDP 67/68 не публикуются, рабочие интерфейсы хоста не подключаются. Privileged нужен для этих внутренних сетевых операций. Не запускайте Python-сценарий напрямую на рабочем Linux-хосте.

Проверяются DORA с broadcast-флагом и без него, unicast-продление с увеличением срока в БД, автоматическое продление ISC dhclient по T1, выбор пула и изоляция VLAN, явное имя VLAN-интерфейса, ISC relay и точное эхо Option 82, корректный NAK, игнорирование BOOTREPLY, сохранение RELEASE с повторным получением аренды и карантин адреса после DECLINE с выдачей другого IP. После перезапуска процесса сверяется БД и отсутствие повторной выдачи занятого адреса. Затем первый контейнер удаляется, создаётся новый с прежним каталогом SQLite и проверяется продление старого клиента и выдача новому.

Результаты, конфиг, SQLite, журналы и захваты пакетов сохраняются в `output/dhcp-lab/`: `report.json` содержит 20 проверок, `recreated-report.json` - проверку нового контейнера. `vlan-wire.pcap` подтверждает реальные теги 802.1Q и PCP 6/5, `relay-wire.pcap` - обмен с relay и Option 82; отдельные PCAP показывают обмены клиентов. При сбое команда завершается ненулевым кодом. Повторный запуск очищает только тестовую БД и состояние dhclient в этом каталоге. Стенд не проверяет systemd как PID 1, PostgreSQL и физические коммутаторы.

Проверка systemd с настоящей перезагрузкой Linux

Для проверки собранных пакетов из исходного проекта выполните:

```
bash scripts/test-systemd-vm.sh
```

Сначала должны существовать пакеты текущей версии в `dist/releases/<version>/`. Нужны Go для их сборки, Docker, Python 3 на машине запуска и интернет при подготовке образов. Скрипт использует QEMU ARM64 с программной эмуляцией, поэтому запускается и на amd64 без KVM. Диску Docker потребуется несколько гигабайт, проверка длится несколько минут. Для одного дистрибутива передайте `debian` или `rocky`.

Стенд создаёт отдельные полноценные виртуальные машины из минимальных Debian 12 и Rocky Linux 9 с их собственными пакетами ядра и systemd PID 1. Виртуальных внешних сетевых карт нет; DHCP-клиент подключён внутренним veth. После первой серии проверок внутри гостя выполняется `systemctl reboot`: смена kernel boot_id подтверждает настоящую перезагрузку ядра, затем systemd автоматически запускает включённую службу. Это не замена перезагрузки рестартом процесса или контейнера.

На каждом дистрибутиве проверяются пользователь godhcp, единственная capability для UDP 67, NoNewPrivileges, права env/SQLite/YAML, DORA через пакетный unit, изменение DNS через reload без смены PID, отказ reload при неправильном YAML, ошибка запуска при занятом UDP 67, сохранение аренд при рестарте службы, автозапуск после перезагрузки и продление прежней аренды без повторной выдачи занятого адреса. Для старта, reload и управления службой используйте root или разрешённый sudo; сетевой процесс root не требует.

Отчёты находятся в `output/systemd-lab/debian/godhcp-lab/report.json` и `output/systemd-lab/rocky/godhcp-lab/report.json`; рядом сохранены журналы обоих запусков, свойства unit, kernel boot_id и PCAP. Ненулевой код означает сбой проверки. Образы стенда минимальные: SELinux enforcing, firewalld, реальные NIC и загрузка физического сервера отдельно не моделируются.

Ежедневная проверка и диагностика

- Служба активна, UDP 67 принадлежит GoDHCP, в журнале нет `dhcp send`, `persist lease`, `pool allocate failed`, ошибок БД и bind. При `persist lease` проверьте доступность БД, место на диске и права: OFFER/ACK для незафиксированной аренды теперь не отправляется. Когда чтение работает, пул восстанавливается из БД; если недоступны и чтение, и запись, занятость сохраняется консервативно до сверки. После восстановления БД выполните проверку и reload, чтобы убрать временно удержанные адреса.
- На UI проверьте пулы и аренды; значение занятости включает резервации и удержанные адреса и не равно числу подключённых устройств.
- Метрики доступны на `http://127.0.0.1:9090/metrics`. Следите за исчерпанием пулов, отклонёнными relay и ошибками разбора.
- Если нет OFFER, проверьте firewall, интерфейс/VLAN, наличие подсети и доступность UDP 67. Если нет ACK, сверяйте запрошенный адрес, server-id и состояние аренды.
- Для relay проверьте `giaddr`, маршруты в обе стороны, trusted relay, Option 82, hops и формат Circuit ID. Не включайте `default-pool` для неизвестных VLAN без явной необходимости.
- Удаление пользователя пока не отзывает выданные JWT; refresh-токен сохраняет прежнюю роль. При срочном отзыве доступа смените JWT secret и перезапустите службу; это завершит все JWT-сеансы. API-ключи отзываются удалением из конфигурации.

Дополнительно: параметры YAML (configuration.md), REST API (api.md), VLAN (vlans.md), relay (relay.md), формат импорта (import-dhcpd.md).

Лицензия и автор

GoDHCP распространяется по Apache License 2.0. Автор и правообладатель собственного кода: Кислов Роман Сергеевич, 2026. В пакете лицензия и NOTICE находятся в `/usr/share/licenses/godhcp/`, там же каталог `third_party` с лицензиями зависимостей. Сторонние компоненты сохраняют своих авторов и условия; на них авторство GoDHCP не распространяется. В самостоятельный PDF вложены LICENSE, NOTICE и уведомления лицензии DejaVu для встроенных шрифтов.

После обновления с unit версии 0.1.2 замените ручной unit, если он есть в `/etc/systemd/system/`, выполните `systemctl daemon-reload` и `systemctl restart godhcp` в согласованное окно работ: старый работающий процесс ещё не предоставляет новый управляющий сокет reload.

Обновление до v0.1.8 и повторный импорт

Полная пошаговая инструкция по обновлению (upgrade-v0.1.8.md) описывает резервную копию настроек и базы, установку пакета, повторный импорт, перенос резерваций, проверку через панель и откат. Обновление выполняется на сервере; браузер не устанавливает бинарник. В PDF этот раздел включён полностью.

© 2026 Кислов Роман Сергеевич. Apache License 2.0. Редакция 31 октября 2026.

Инструкция для уже работающего GoDHCP с SQLite и пакетной службой systemd. Команды выполняйте в root-сессии Bash (`sudo -i`) в согласованное окно обслуживания. Существующие клиенты сохраняют полученные адреса до истечения своих аренд; во время остановки сервера новые выдачи и продления недоступны.

База аренд остаётся прежней. Импорт выполняется в отдельный YAML, из которого вручную переносятся только нужные настройки. Импортированный файл не заменяет рабочую конфигурацию целиком.

Если DHCP пока обслуживает ISC dhcpd, эта инструкция не заменяет план миграции: GoDHCP не импортирует состояние из `dhcpd.leases`. Занятые адреса ISC должны быть защищены от повторной выдачи отдельно.

1. Проверьте версию и действующие пути

```
godhcp version
systemctl cat godhcp
less /etc/godhcp/config.yml
```

Далее предполагаются бинарник `/usr/bin/godhcp`, конфиг `/etc/godhcp/config.yml`, секреты `/etc/godhcp/godhcp.env`, база `/var/lib/godhcp/leases.db`. Проверьте путь запуска в unit и значение `database.dsn`. Если пути отличаются, замените их во всех командах, включая резервное копирование. Для PostgreSQL нужна резервная копия средствами PostgreSQL; команды копирования SQLite к нему не применяются.

Не публикуйте конфиг и env-файл: они могут содержать пароли, JWT и API-ключи. Экспорт из веб-интерфейса маскирует часть секретов и не заменяет резервную копию файлов сервера.

2. Скачайте пакет и проверьте SHA-256

Скачайте из релиза v0.1.8 файл `SHA256SUMS` и один пакет своей системы в отдельный каталог. Проверьте архитектуру:

```
uname -m
```

Следующие пакеты предназначены для `x86_64`:

- ALT Linux p10: `godhcp-0.1.8-alt1.p10.x86_64.rpm`.
- Astra Linux: `godhcp_0.1.8-1astra_amd64.deb`.
- РЕД ОС: `godhcp-0.1.8-1.redos.x86_64.rpm`.

В каталоге скачивания выполните только одну подходящую проверку:

```
# ALT Linux p10:
grep '  godhcp-0.1.8-alt1.p10.x86_64.rpm$' SHA256SUMS \
  | sha256sum -c -

# Astra Linux:
grep '  godhcp_0.1.8-1astra_amd64.deb$' SHA256SUMS \
  | sha256sum -c -

# РЕД ОС:
grep '  godhcp-0.1.8-1.redos.x86_64.rpm$' SHA256SUMS \
  | sha256sum -c -
```

Продолжайте только при результате `OK`. SHA-256 проверяет целостность; пакеты не подписаны ключом репозитория. RPM для РЕД ОС проверен на Rocky Linux 9, а не на самой РЕД ОС: официальный реестр не прошёл проверку TLS-сертификата.

3. Остановите службу и сохраните настройки и аренды

На время работ прекратите изменения через веб-интерфейс. Сначала остановите службу:

```
systemctl stop godhcp
systemctl is-active godhcp
```

Ожидается `inactive`; команда проверки при этом возвращает ненулевой код. Если служба остаётся активной, не переходите к копированию базы: выясните причину. Дальнейшие команды выполняйте вручную, а не единым скриптом с `set -e`.

В той же root-сессии Bash создайте резервную копию:

```
umask 077
backup_dir="/root/godhcp-backup-$(date +%Y%m%d-%H%M%S)"
install -d -m 0700 "$backup_dir"

cp -a /etc/godhcp "$backup_dir/etc-godhcp"
cp -a /var/lib/godhcp "$backup_dir/var-lib-godhcp"
cp -a /usr/bin/godhcp "$backup_dir/godhcp.previous"
systemctl cat godhcp > "$backup_dir/godhcp.service.txt"
printf 'Резервная копия: %s\n' "$backup_dir"
```

Запишите выведенный путь: переменная `backup_dir` не сохраняется после закрытия сессии. Если база расположена вне `/var/lib/godhcp`, скопируйте после остановки и её каталог с сохранением прав.

Копируется весь каталог базы вместе с возможными `leases.db-wal` и `leases.db-shm`. Не удаляйте их отдельно и не копируйте один `leases.db` во время работы сервера. Сохраните также внешние сертификаты и секреты, расположенные вне `/etc/godhcp`, если используете их.

Если установлен `sqlite3`, проверьте копию:

```
sqlite3 "$backup_dir/var-lib-godhcp/leases.db" \
  'PRAGMA integrity_check;'
```

Ожидается `ok`. При ошибке не продолжайте обновление до выяснения причины. Храните резервную копию с ограниченным доступом.

4. Установите обновление

В каталоге скачанного пакета выполните одну команду своей системы:

```
# ALT Linux p10:
apt-get install ./godhcp-0.1.8-alt1.p10.x86_64.rpm

# Astra Linux:
apt install ./godhcp_0.1.8-1astra_amd64.deb

# РЕД ОС:
dnf install ./godhcp-0.1.8-1.redos.x86_64.rpm
```

Затем проверьте версию:

```
systemctl daemon-reload
/usr/bin/godhcp version
```

Должно отобразиться `godhcp 0.1.8`. Не копируйте `config.yml.example` поверх рабочего конфига, не создавайте новую базу и не удаляйте старую. Пакеты не запускают службу автоматически. При обновлении вручную установленного бинарника сохраните прежний unit и пути запуска; сначала проверьте, что новый пакет не конфликтует с вашей схемой установки.

5. Повторите импорт в отдельный файл

Исходный `dhcpd.conf` и все файлы из директив `include` должны быть доступны по указанным путям. Относительные пути разрешаются от файла, содержащего include. Следующая команда перезаписывает только указанные файлы результата и журнала, если они уже существуют:

```
umask 077
/usr/bin/godhcp import-dhcpd \
  -in /etc/dhcp/dhcpd.conf \
  -out /etc/godhcp/imported-v018.yml \
  2> /etc/godhcp/imported-v018.log
```

Проверьте, что команда завершилась без ошибки, затем прочитайте журнал и YAML:

```
cat /etc/godhcp/imported-v018.log
less /etc/godhcp/imported-v018.yml
```

Импорт не запускает DHCP, не меняет исходный `dhcpd.conf` и не записывает аренды в базу. В версии 0.1.7 исправлена потеря `shared-network` после неподдерживаемого блока `class`. Для примера VLAN116 должны сохраниться подсеть `10.0.116.0/22` и фиксированные адреса, включая `SRV-ZABBIX` с `10.0.116.10`, если они есть в исходнике.

Предупреждение о пропуске `class` ожидаемо. Условия классов не переносятся автоматически: выбор подсети по relay (`giaddr` или Option 82) необходимо проверить вручную. Директива `authothriative` в исходнике содержит опечатку; правильное написание `authoritative`, но менять режим без проверки роли сервера не следует.

6. Перенесите недостающие резервации в копию рабочего конфига

```
cp -a /etc/godhcp/config.yml /etc/godhcp/config.next.yml
```

Откройте `config.next.yml` в доступном текстовом редакторе. Из `imported-v018.yml` перенесите нужные записи `reservations` в соответствующие существующие подсети. Пример фрагмента внутри подсети:

```
    reservations:
      - mac: "78:e3:b5:14:dc:7c"
        ip: 10.0.116.10
        hostname: SRV-ZABBIX
```

Если список `reservations` уже есть, добавьте элементы в него; не создавайте второй одноимённый ключ. Соблюдайте отступы YAML и не используйте табуляцию. Сверьте число и адреса резерваций с исходными `host`/`fixed-address`, проверьте дубликаты MAC/IP и конфликты с действующими арендами других клиентов. При конфликте согласуйте перенос клиента; не

удаляйте активную аренду для принудительной выдачи занятого адреса.

Сохраните текущие `database.driver`, `database.dsn`, идентификаторы подсетей `id`, диапазоны, интерфейсы, VLAN, relay, параметры API, пользователей, TLS и секреты. Новые идентификаторы из импорта не должны заменять существующие: с ними связаны текущие аренды. Если подсеть ранее отсутствовала, отдельно перенесите и проверьте её параметры и выбор через relay.

Импортированный файл не является полной копией настроенного GoDHCP: в частности, в нём не восстановлены ваши пользователи и политики доступа. Применение всего результата импорта поверх рабочего конфига может изменить доступ к управлению.

7. Проверьте кандидат и запустите сервер

Для пакетной службы следующая команда проверяет конфиг от пользователя `godhcp`, загружая секреты из того же env-файла, что использует служба:

```
systemd-run --unit=godhcp-validate-v018 --wait --pipe \
  -p User=godhcp \
  -p Group=godhcp \
  -p EnvironmentFile=/etc/godhcp/godhcp.env \
  /usr/bin/godhcp -validate \
  -config /etc/godhcp/config.next.yml
```

Ожидается `configuration is valid` и успешное завершение команды. При ошибке рабочий файл не меняйте. Проверьте права чтения конфига для `godhcp`, наличие env-файла и значения переменных; обычная shell-команда сама `godhcp.env` не загружает. При повторном запуске после неудачной проверки можно использовать новое имя unit, например `godhcp-validate-v018-retry`.

После успешной проверки:

```
cp -a /etc/godhcp/config.next.yml /etc/godhcp/config.yml
systemctl start godhcp
systemctl status godhcp --no-pager
journalctl -u godhcp -n 100 --no-pager
```

Если запуск не удался, не повторяйте его многократно: прочитайте ошибку и используйте копию конфигурации для отката. DHCPv4 использует стандартный UDP 67; порт должен быть свободен, а процессу нужны соответствующие привилегии. При `address already in use` выполните `ss -lunp` и выясните владельца порта, прежде чем останавливать другой DHCP-сервер. Существующий адрес API и TLS сохраняются; порт 8181 относится к новым примерам конфигурации, а не к обязательной смене вашего порта.

8. Проверьте результат в веб-интерфейсе

Откройте прежний URL сервера и войдите с прежней учётной записью. Если вход перестал работать, проверьте, не заменили ли вы настройки `api.auth` импортированными значениями.

- В разделе «Аренды» найдите несколько ранее работавших клиентов: их IP и активные аренды должны остаться.
- В разделе «Резервации» найдите добавленные MAC и проверьте IP и подсеть.
- На одном согласованном тестовом устройстве обновите DHCP-адрес и проверьте получение зарезервированного IP.
- В разделе «Журнал» проверьте отсутствие ошибок базы, выбора подсети и отправки ответов.

Не перезапускайте все клиентские устройства одновременно. Сохраните резервную копию и журнал импорта до завершения проверки. Скриншоты разделов панели приведены в иллюстрированном руководстве (web-guide.md).

9. Откат при проблеме

Остановите службу. Восстановите конфиг из записанного каталога резервной копии; если сессия была закрыта, сначала задайте `backup_dir` фактическим путём вашей копии.

```
systemctl stop godhcp
cp -a "$backup_dir/etc-godhcp/config.yml" \
  /etc/godhcp/config.yml
```

Если меняли env-файл или unit, восстановите и их. Для отката версии предпочтительно установить сохранённый предыдущий пакет. Затем проверьте конфигурацию и запустите службу. Отдельная копия прежнего бинарника `godhcp.previous` сохранена для аварийного восстановления; ручная замена файла требует согласования с установленным пакетом.

Текущую базу аренд сохраняйте. После запуска v0.1.8 могли появиться новые аренды. Восстановление старой копии базы не делает эти адреса свободными и может привести к повторной выдаче. Восстановление базы выполняйте только отдельной процедурой с учётом всех выдач после снимка; не смешивайте старые и новые WAL/SHM-файлы.